

Міністерство освіти та науки України

Національний університет «Львівська політехніка»

Інститут телекомунікацій, радіоелектроніки та електронної техніки



ЗВІТ

З дисципліни

“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
Студент групи ІХ-22
Брона М.В.

Прийняв:
Гордійчук-Бублівська О.В

Львів 2026

Тема: Реалізація клієнт-серверної взаємодії з використанням REST API.

Мета: Розроблення клієнт-серверної архітектури з використанням REST API та практичне застосування класів для реалізації взаємодії між компонентами системи.

```
=====
JSCraftBuilder API (https://jscraftbuilder.typicode.com)
=====

[1] GET /posts - отримати всі пости:
GET OK (status: 200)
Отримано 100 постів. Відповідь: {"userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"}

[2] GET /posts/1 - отримати пост #1:
GET OK (status: 200)
Знайдено пост aut facere repellat provident occaecati excepturi optio reprehenderit.

[3] GET /posts/1/comments - коментарі до поста #1:
GET OK (status: 200)
Знайдено 5 коментарів.

[4] POST /posts - створити новий пост:
Created (status: 201): {"userId": 10, "title": "Тест OK! Hello", "body": "Це тестовий пост із лабораторної роботи", "id": 101}

[5] PUT /posts/1 - оновити пост #1:
Post updated (status: 200)

[6] DELETE /posts/1 - видалити пост #1:
Post deleted (status: 200)

[7] GET /users - список користувачів:
GET OK (status: 200)
Отримано 10 користувачів.
ID 1: Ismael Ibrahim (ismacere@er11.biz)
ID 2: Ervin Hamill (sharad@melissa.tv)
ID 3: Clementine Koch (harlow@rosalia.net)

=====
Open Metro API (New API schema)
https://open-metro.com/
=====

Місто: Lviv
Координати: 49.8383, 24.9232 (Україна)
Температура: 13.2°C (відхиляється на 12.2°C)
Humidity: 78%
Wind: 983.9 r/s
Windp: 5.8 m/s

Місто: London
Координати: 51.5085, 0.1257 (Велика Британія)
Температура: 15.7°C (відхиляється на 14.2°C)
Humidity: 88%
Wind: 1054.9 r/s
Windp: 18.4 m/s

Місто: Kyiv
Координати: 50.4547, 30.5238 (Україна)
Температура: 21.4°C (відхиляється на 21.9°C)
Humidity: 65%
Wind: 998.3 r/s
Windp: 3.8 m/s

PS C:\Users\Marko
```

```

import requests
import json

class JsonOperations:
    def __init__(self, filename: str):
        self.filename = filename

    def save_to_json(self, data):
        with open(self.filename, 'w') as f:
            json.dump(data, f, indent=4)

    def read_json(self):
        with open(self.filename, 'r') as f:
            return json.load(f)

class RestClient:
    def __init__(self, url):
        self.url = url

    def _make_request(self, method, endpoint, **kwargs):
        try:
            res = requests.request(method, endpoint, **kwargs)
            res.raise_for_status()
            return res
        except requests.exceptions.HTTPError as e:
            print(f" HTTP error {res.status_code}: {e}")
            return None
        except requests.exceptions.ConnectionError:
            print(" Connection error: не вдалося підключитися до сервера.")
            return None
        except requests.exceptions.Timeout:
            print(" Timeout: сервер не відповів вчасно.")
            return None
        except requests.exceptions.RequestException as e:
            print(f" Request error: {e}")
            return None

    def get(self, resource):
        endpoint = f"{self.url}/{resource}"
        res = self._make_request("GET", endpoint)
        if res:
            print(f" GET OK (status {res.status_code})")
            return res.json()
        return None

    def post(self, resource, data):
        endpoint = f"{self.url}/{resource}"
        res = self._make_request("POST", endpoint, json=data)
        if res:
            print(f" Created (status {res.status_code}):", res.json())
            return res.json() if res else None
        return None

    def put(self, resource, post_id, new_data):
        endpoint = f"{self.url}/{resource}/{post_id}"
        res = self._make_request("PUT", endpoint, json=new_data)
        if res:
            print(f" Post updated (status {res.status_code})")
            return res.json() if res else None

```

```

C:\Users\Marko> python lab6_OOPL.py
19 class RestClient:
20     def put(self, resource, post_id, new_data):
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63     def delete(self, resource, post_id):
64         endpoint = f"{self.url}/{resource}/{post_id}"
65         res = self._make_request("DELETE", endpoint)
66         if res:
67             print(f" Post deleted (status {res.status_code})")
68
69
70 # -----
71 # Частина 1: JSONPlaceholder (не потребує API ключа)
72 # -----
73 def demo_jsonplaceholder():
74     print("=" * 55)
75     print(" JSONPlaceholder API (https://jsonplaceholder.typicode.com)")
76     print("=" * 55)
77
78     url = "https://jsonplaceholder.typicode.com"
79     client = RestClient(url)
80
81     # GET – список постів
82     print("\n[1] GET /posts – отримати всі пости:")
83     posts = client.get("posts")
84     if posts:
85         JsonOperations("posts.json").save_to_json(posts)
86         print(f" Отримано {len(posts)} постів. Перший: {posts[0]}")
87
88     # GET – один пост
89     print("\n[2] GET /posts/1 – отримати пост #1:")
90     post = client.get("posts/1")
91     if post:
92         print(f" Заголовок: {post.get('title')}")
93
94     # GET – коментарі до поста
95     print("\n[3] GET /posts/1/comments – коментарі до поста #1:")
96     comments = client.get("posts/1/comments")
97     if comments:
98         print(f" Знайдено {len(comments)} коментарів")
99
100    # POST – створити новий пост
101    print("\n[4] POST /posts – створити новий пост:")
102    new_post = {
103        "userId": 10,
104        "title": "Тест ООП Лаб6",
105        "body": "Це тестовий пост із лабораторної роботи"
106    }
107    client.post("posts", new_post)
108
109    # PUT – оновити пост
110    print("\n[5] PUT /posts/1 – оновити пост #1:")
111    updated_post = {
112        "userId": 10,
113        "title": "Оновлений заголовок",
114        "body": "Оновлений текст поста"
115    }
116    client.put("posts", 1, updated_post)
117
118    # DELETE – видалити пост
119    print("\n[6] DELETE /posts/1 – видалити пост #1:")
120    client.delete("posts", 1)
121

```

```

C:\Users\Marko > Ла66_ООП_1.py
156 class RestWeatherClient:
179     def get_weather(self, coordinates: tuple) -> dict | None:
180         if not coordinates:
181             return None
182         lat, lon, _ = coordinates
183         params = {
184             "latitude": lat,
185             "longitude": lon,
186             "current": [
187                 "temperature_2m",
188                 "apparent_temperature",
189                 "relative_humidity_2m",
190                 "surface_pressure",
191                 "wind_speed_10m",
192             ],
193         }
194         try:
195             res = requests.get(self.WEATHER_URL, params=params)
196             res.raise_for_status()
197             return res.json()
198         except requests.exceptions.RequestException as e:
199             print(f" Помилка отримання погоди: {e}")
200             return None
201
202
203 def process_city(client: RestWeatherClient, city: str, filename_prefix: str):
204     print(f"\n Місто: {city}")
205     coordinates = client.get_lat_lon(city)
206     if not coordinates:
207         return
208
209     lat, lon, country = coordinates
210     print(f" Координати: {lat:.4f}, {lon:.4f} ({country})")
211
212     weather_data = client.get_weather(coordinates)
213     if not weather_data:
214         return
215
216     # Зберегти повну відповідь
217     json_file = JsonOperations(f'{filename_prefix}.json')
218     json_file.save_to_json(weather_data)
219
220     # Зберегти та вивести скорочену інформацію
221     weather_base = Weather().basic_weather_info(weather_data)
222     json_file.save_to_json(weather_base)
223
224     print(f" Температура : {weather_base['temp']]°C "
225           f" (відчувається як {weather_base['feels_like']]°C)")
226     print(f" Вологість : {weather_base['humidity']]%")
227     print(f" Тиск : {weather_base['pressure']] rPa")
228     print(f" Вітер : {weather_base['wind_speed']] км/г")
229
230
231 def demo_weather():
232     print("\n" + "=" * 55)
233     print(" Open-Meteo API (без API ключа)")
234     print(" https://open-meteo.com/")
235     print("=" * 55)
236
237     client = RestWeatherClient()
238     for city, prefix in [("Lviv", "weatherLviv"),
239                        ("London", "weatherLondon"),
240                        ("Kyiv", "weatherKyiv")]:
241         process_city(client, city, prefix)
242
243
244 # -----
245 def main():
246     demo_jsonplaceholder()
247     demo_weather()
248
249
250 if __name__ == "__main__":
251     main()
252

```

Висновок:

Під час виконання лабораторної роботи я спроектував клієнт-серверну архітектуру з використанням принципів REST API. Практична частина роботи дозволила закріпити навички використання об'єктно-орієнтованого підходу для реалізації взаємодії між окремими компонентами системи

